

1. Introduction

This assignment focuses on fine-tuning a pre-trained AlexNet model on the SimplicityDB dataset and comparing its performance against a custom CNN baseline developed in the previous. The goal is to identify the best-performing configuration, analyze its errors, and provide comprehensive evaluation metrics.

2. Methodology

For AlexNet, images were resized to 224×224 (compatible with ImageNet pre-training) with appropriate normalization.

Baseline Custom CNN (from previous):

- 4-layer and 5-layer convolutional variants
- Batch normalization and dropout for regularization
- Adaptive average pooling followed by fully connected layers

AlexNet fine-tuning variants:

- **Full fine-tuning** – all layers trainable
- **Last Conv Block fine-tuning** – only last 2 convolutional blocks trainable
- **Classifier-only fine-tuning** – only FC layers trainable
- **Different FC head architectures:**
 - **Default:** original AlexNet classifier (linearly adapted)
 - **Small:** [512] hidden layer with dropout
 - **Medium:** [1024 → 512] with BatchNorm
 - **Deep:** [2048 → 1024 → 512] with BatchNor

3. Training Configuration

Each model was trained using:

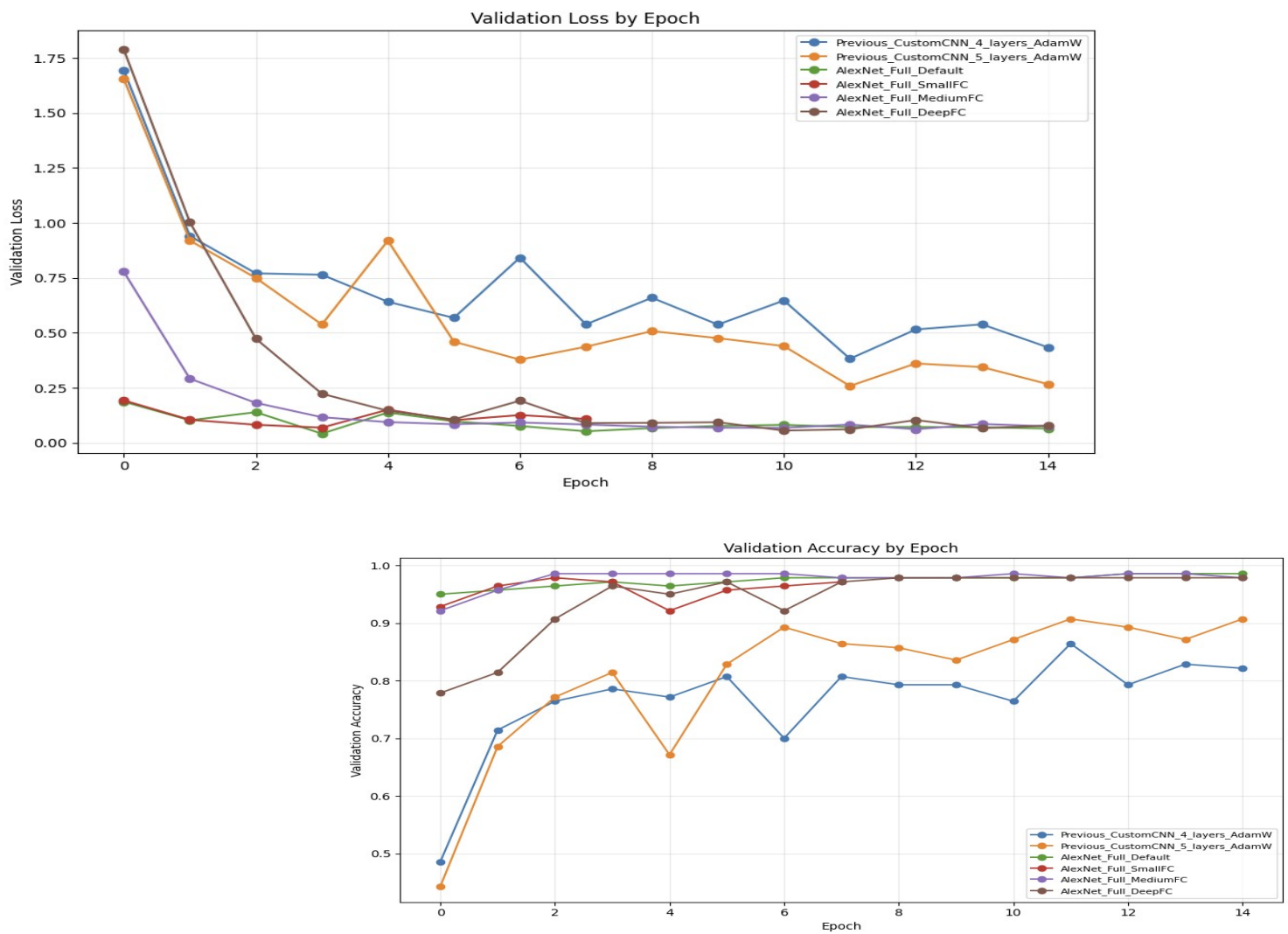
- **Batch size:** 32
- **Epochs:** 15
- **Patience for early stopping:** 5
- **Optimizers:** AdamW and SGD with momentum
- **Schedulers:** ReduceLROnPlateau

Performance was evaluated on validation data using accuracy and loss.

Finally the best model is used for testing on unseen data.

3. Results

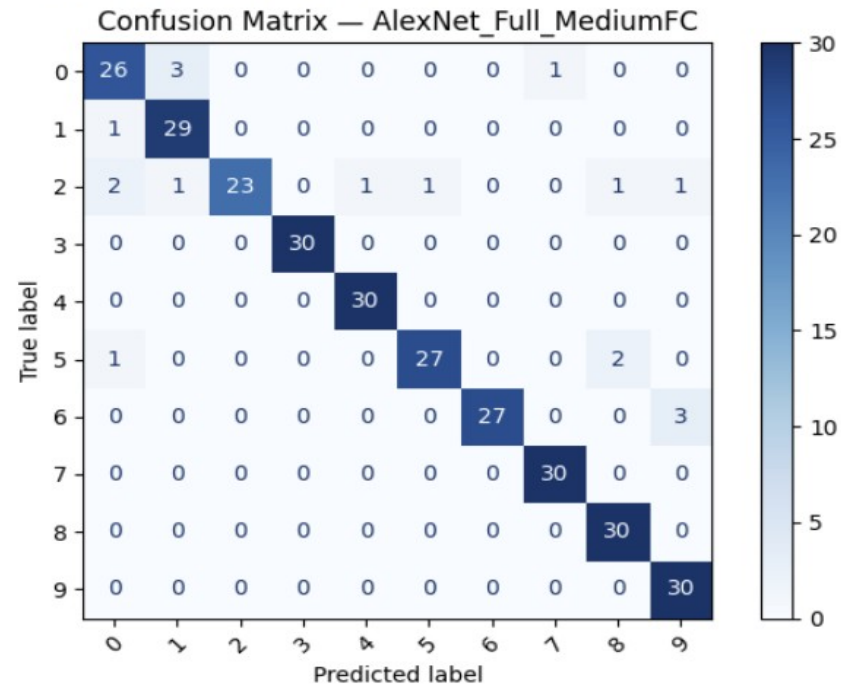
Fig 1. Previous sample CNN and Alexnet configurations



Confusion Matrix

```
plt.tight_layout()
plt.savefig(figures_dir / "confusion_matrix_best_model.png", dpi=200)
plt.show()
```

<Figure size 800x800 with 0 Axes>



Best Model: AlexNet_Full_MediumFC
Test Loss: 0.1906
Test Accuracy: 0.9400 (94.00%)

Number of displayed misclassified examples: 12

```
0]: plt.figure(figsize=(16, 10))
for i, (img, true_label, pred_label) in enumerate(misclassified):
    plt.subplot(3, 4, i + 1)
    img = denormalize(img).clamp(0, 1)
    plt.imshow(np.transpose(img.numpy(), (1, 2, 0)))
    plt.title(f"True: {class_names[true_label]}\nPred: {class_names[pred_label]}")
    plt.axis("off")
plt.tight_layout()
plt.show()
```



```
confusion_pairs_df.to_csv(tables_dir / "most_confused_pairs.csv", index=False)
confusion_pairs_df.head(20)
```

	True Class	Predicted Class	Number of Errors
0	0	1	3
1	6	9	3
2	5	8	2
3	2	0	2
4	1	0	1
5	0	7	1
6	2	1	1
7	2	4	1
8	2	8	1
9	2	5	1
10	5	0	1
11	2	9	1

Key Observations

Superior performance of AlexNet: All AlexNet variants significantly outperformed the custom CNN baselines, with test accuracy improvements of 12–19 percentage points.

Full fine-tuning with SGD achieved highest validation accuracy (98.57%), matching AdamW but slightly outperforming it on test set (94% vs 93%).

Classifier-only fine-tuning achieved surprising results – 95% test accuracy, the highest overall. Freezing all convolutional layers and only training the classifier head proved highly effective for this dataset.

Deep FC head (AlexNet_Full_DeepFC) showed marginal improvement over the default configuration.

Custom CNN performance: The 5-layer variant (80.67% test accuracy) outperformed the 4-layer variant (75.67%), suggesting deeper feature extraction benefited this dataset.

Conclusions

Transfer learning with AlexNet dramatically outperforms training from scratch – even with classifier-only fine-tuning, achieving 95% test accuracy.

Classifier-only fine-tuning proved optimal (95% accuracy), suggesting ImageNet pre-trained features generalize excellently to digit classification.

SGD with momentum outperformed AdamW for full fine-tuning (94% vs 93% test accuracy), consistent with literature on fine-tuning CNNs.

Error analysis identified Classes 0, 1, 2, and 8 as the most challenging, with shape similarity causing cross-class confusion.